

系统开发工具基础·实验报告（第 5–8 题）

姓名：李云龙 学号：25030021043

2026 年 8 月 31 日

姓名	李云龙
学号	25030021043
实验日期	2026 年 8 月 31 日
实验环境	Windows 11 + WSL2 (Ubuntu 26.04 LTS); bash; git; Python 3.14 + pytest 9.1 + ruff 0.16

说明：本报告中各题“运行结果”均为在 WSL 中实际运行的输出；第 5 题的 Ctrl-Z / bg / jobs 等为终端交互操作，其产物（日志文件）与关键输出一并列出。

1 实验目的

1. 掌握进程信号（SIGTERM/SIGINT）、trap 清理机制与 Shell 任务控制（挂起/后台/取 PID）。
2. 掌握编辑器语言服务器的跳转定义、查找引用与语义重命名，以及 ruff/pytest 的开发反馈闭环。
3. 掌握用调试器（pdb/IDE）定位逻辑缺陷的方法与最小修复原则。
4. 掌握“先测量再优化”的性能分析方法（time / cProfile）与公平对比（同输入、多次取中位数）。

2 实验五：控制一个可清理的后台任务（q05）

2.1 完整脚本（q05/worker.sh）

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
```

赋予执行权限，前台启动并把 stdout/stderr 分别写入日志：

```
cd q05
chmod +x worker.sh
./worker.sh > stdout.log 2> stderr.log
```

2.2 交互控制步骤

1. **Ctrl-Z** 挂起任务（前台进程转入停止状态，Shell 提示 [1]+ Stopped）；
2. **bg** 让任务在后台继续运行；
3. **jobs** 确认状态：[1]+ Running；
4. **pid=\$(jobs -p)** 取得任务 PID（由 Shell 管理，不手工抄写）；
5. **kill -TERM "\$pid"** 发送 SIGTERM；**wait** 等待任务结束；
6. **cat cleanup.log** 确认出现 CLEAN_EXIT。

2.3 运行结果

cleanup.log:

```
CLEAN_EXIT
```

stdout.log (73 行，数字 0-72，头尾示例):

```
0
1
...
71
72
```

stderr.log: 为空（程序没有错误输出）。

2.4 要点分析

- **trap '...' TERM INT** 为进程注册信号处理:收到 SIGTERM/SIGINT 时先执行清理(追加 CLEAN_EXIT 日志)再 **exit 0**, 实现**优雅退出**；**kill -9** 由内核直接强杀，信号不可捕获，清理代码永远不会执行，故题目禁止。
- **Ctrl-Z / bg / jobs / jobs -p** 构成 Shell **任务控制**标准链路: 挂起 -> 后台 -> 查状态 -> 取 PID。用 **jobs -p** 而非手抄 PID，避免抄错或拿到其它进程的 PID。
- **stdout** 与 **stderr** 分离重定向便于日志分类排查: 正常输出进 **stdout.log**, 错误信息进 **stderr.log**。

3 实验六：语义重构与本地开发反馈（q06）

3.1 文件与最终代码

math_utils.py:

```
def calculate_total(price: float, count: int) -> float:
    return price * count
```

app.py:

```
from math_utils import calculate_total

print(calculate_total(12.5, 4))
```

test_math_utils.py:

```
from math_utils import calculate_total

def test_calculate_total():
    assert calculate_total(12.5, 4) == 50.0
```

3.2 语言服务器演示步骤（VS Code + Pylance / VS + Python 扩展）

1. **跳转到定义**：在 app.py 的 `calculate_total(...)` 上按 F12，跳到 math_utils.py 中的定义；
2. **查找引用**：Shift+F12，列出定义与全部使用位置（app.py 的调用、测试中的调用）；
3. **重命名符号**：F2 输入 `calculate_total`，语言服务器按符号身份同步修改定义与所有引用（本项目 3 处）；
4. **ruff 反馈**：临时在 app.py 加入 `import os`（未使用）-> `ruff check .` 报 F401 unused import -> 删除该行 -> 重新检查通过。

3.3 运行结果

```
$ ruff check .
All checks passed!

$ python3 -m pytest -q
. [100%]
1 passed in 0.07s
```

3.4 要点分析

- 语言服务器的重命名是**语义级**操作（按符号身份定位），不是文本替换，不会漏改引用、也不会误改注释或字符串中恰好相同的名字；
- ruff 是静态检查工具，无需运行程序即可发现未使用导入（F401）、未定义变量等“代码异味”，与语言服务器、pytest 一起构成“编辑 -> 检查 -> 测试”的本地开发反馈闭环。

4 实验七：用调试器定位归并排序缺陷（q07）

4.1 缺陷代码（merge_sort_buggy.py，题目给定）

4.5 要点分析

- 先运行确认“结果错误”，再用调试器定点观察 i 、 j 、 $left[i]$ 、 $right[j]$ 四个变量，指针混用立刻现形——比到处加 `print` 更高效；
- **最小修复原则**：只改出错的一处下标，保持算法结构不变，并用两个测试（含重复元素）做回归保护。

5 实验八：先测量，再优化慢速词频程序（q08）

5.1 文件

`generate_words.py`（可重复生成词表，固定随机种子）：

```
import random

random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
```

`wordfreq.py`（原版，去重用 `list + in`）：

```
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
```

`wordfreq_fast.py`（优化版，`set` 去重）：

```
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
```

5.2 测量与分析方法

```
cd q08
python3 generate_words.py           # 生成 words.txt (30000 词)
time python3 wordfreq.py            # 原版，跑 2 次
python3 -m cProfile -s cumulative wordfreq.py  # 热点分析
time python3 wordfreq_fast.py       # 优化版，跑 2 次
```

cProfile 结果：热点集中在 `wordfreq.py:1` 的去重循环（`list` 的 `in` 成员判断，`tottime` 0.074s），而非文件读取/分词。

5.3 实测数据（同一 words.txt，各跑 2 次）

版本	run1	run2	中位数
原版 (list + in)	0.099s	0.093s	0.096s
优化版 (set 去重)	0.026s	0.019s	0.023s

加速比 $\approx 0.096/0.023 \approx 4.3$ 倍；最终 count 均为 1000，保持不变（未写死词表大小）。

5.4 要点分析

- **先测量再优化**：cProfile 证明热点是去重循环的成员判断，而不是文件读取，避免优化错方向；
- 复杂度：list 的 in 是线性扫描（整体 $O(n^2)$ ），set 基于哈希表（整体 $O(n)$ ）；
- 对比方法：**相同输入**、每版本跑 2 次取中位数再算加速比，抵消系统噪声，结论更可信。

6 遇到的问题与解决

问题	原因	解决
第 7 题归并结果错乱	merge 的 else 分支 right[i] 下标混用（应为 right[j]）	pdb 断点观察指针后一行修复，测试回归
第 8 题原版去重慢	list 的 in 为线性扫描，整体 $O(n^2)$	改用 set 去重 ($O(n)$)，count 不变
第 6 题未使用导入检查	import os 未被使用	用 ruff 静态检查发现 F401 并删除
环境缺少 ruff	Ubuntu 26.04 的 apt 无 ruff 包	pip3 install --break-system-packages ruff pytest

7 实验总结

通过本次实验，我掌握了进程信号与 trap 优雅退出机制，以及 Ctrl-Z / bg / jobs / jobs -p 的任务控制链路；体验了编辑器语言服务器的跳转定义、查找引用与语义重命名，并用 ruff + pytest 建立了本地开发反馈闭环；学会用调试器定点观察变量来定位“指针/下标混用”类缺陷并做最小修复；理解了“先测量再优化”的分析方法，通过 cProfile 定位热点后把去重从 $O(n^2)$ 优化到 $O(n)$ ，实测加速约 4.3 倍，并学会了用中位数做公平的性能对比。

附录：文件清单

w2/README.md	第 5--8 题说明（步骤、原理、实测数据）
w2/q05/worker.sh	第 5 题脚本（trap TERM/INT 清理）
w2/q05/cleanup.log / stdout.log / stderr.log	第 5 题运行产物
w2/q06/math_utils.py / app.py / test_math_utils.py	第 6 题（已重命名 calculate_total）
w2/q07/merge_sort_buggy.py	第 7 题缺陷版
w2/q07/merge_sort.py	第 7 题修复版

w2/q07/test_merge_sort.py 第 7 题测试 (2 个用例)

w2/q08/generate_words.py / wordfreq.py / wordfreq_fast.py / words.txt 第 8 题

实验报告.md / 实验报告.tex / 实验报告.pdf 本报告